

Chapter : 12

Introducing Abstract Window Toolkit

12.0	Objectives
12.1	Introduction
12.2	Fundamentals
12.3	Frame Windows
12.4	Working with Graphics
12.4.1	Introduction
12.4.2	Drawing Lines
12.4.3	Drawing Rectangle
12.4.4	Drawing Ellipses and Circles
12.5	Color
12.6	Fonts
12.7	Summary
12.8	Check Your Progress - Answers
12.9	Questions for Self - Study
12.10	Suggested Readings

12.0 OBJECTIVES

Dear Students,

After studying this chapter you will be able to :

- discuss what is the Abstract Window Toolkit
- discuss how to create frames in applets
- discuss how to draw lines, circles etc.
- to obtain basic knowledge of colors and the **Font** class

12.1 INTRODUCTION

The **AWT** (Abstract Window Toolkit) contains numerous classes and methods which allow you to create and manage windows. In this chapter you will learn how to create and manage windows, manage fonts, output text and utilise graphics. The next chapter will describe the various controls like push buttons and scroll bars supported by AWT. Remember that though the main purpose of the AWT is to support applet windows, it can also be used to create stand alone windows that run in a GUI environment. The AWT classes are contained in the **java.awt** package. It is one of the largest packages of Java. We shall study some of these classes in this and the next chapter. Remember however, that a brief introduction of the various classes and methods only has been presented. A detailed study is not in the scope of this study material.

12.2 FUNDAMENTALS

The AWT defines windows according to a class hierarchy that adds functionality and specificity with each level. The two most common windows are those derived from **Panel**, which is used by applets and those derived from **Frame** which creates a standard window. At the top of the AWT hierarchy is the **Component** class. This is an abstract class which encapsulates all of the attributes of a visual component. All those user interfaces which are displayed on the screen and those which interact with the user are subclasses of **Component**. **Component** defines over a hundred public methods which are useful for managing events like mouse and keyboard inputs, etc. The **Container** class is a subclass of **Component**. It has additional methods which allow other **Component** objects to be nested within it. Other container objects can be stored inside a **Container**, since they themselves are instances of **Component**. A container is responsible for laying out any components that it contains. This is done through the use of **layout managers**.

The **Panel** class is a concrete subclass of **Container**. It simply implements the **Container**. It does not add any new methods. **Panel** is the superclass for **Applet**. When screen output is directed to an applet, it is drawn on the surface of a panel object. A **Panel** is a window which does not contain a title bar, a menu bar or a border. This is the reason you don't see these items when an applet is run inside a browser. However, when you run an applet in an applet viewer, the applet viewer provides the title and the border. Other components can be added to the **Panel** object by using the **add()** method which is inherited from **Container**.

The **Window** class creates a top level window. A top level window is a window which is not contained within any other object; it sits directly on the desktop. Generally, you don't create a **Window** object directly. You use a subclass of **Window** called **Frame**.

Frame is a subclass of **Window** and has a title bar, menu bar,

12.1 & 12.2 Check Your Progress.

1. Fill in the blanks.

- a) The AWT classes are contained in the package.
- b) The two most common windows are those derived from and those derived from
- c) A is responsible for laying out any components that it contains.

2. Write True or False.

- a) A **Panel** is a window which contains a title bar, a menu bar or a border.
.....
- b) **Canvas** is a part of the hierarchy for applet or frame windows.
.....

borders and resizing corners. If you create a **Frame** object from within an applet, it will contain a warning message (eg. Warning : Applet Window) to the user indicating that an applet window has been created. This message warns the user that the window was started by an applet and not by software running on their computer.

Canvas encapsulates a blank window upon which you can draw. Remember **Canvas** is not a part of the hierarchy for applet or frame windows.

12.3 FRAME WINDOWS

Frame is used to create child windows within applets and top level or child windows for applications. It creates a standard style window. **Frame** supports the following constructors :

`Frame()`

`Frame(String title)`

The first form creates a window which does not contain a title while the second form creates a window with the title specified by *title*. Remember that you cannot specify the dimensions for the window, you must set the size of the window after it has been created. Some of the methods we shall use when working with **Frame** windows include:

The **setSize()** method is used to set the dimensions of the window. It takes the following form :

`void setSize(int newWidth, int newHeight)`

`void setSize(Dimension newSize)`

newWidth and *newSize* specify the size of the window in the first case. In the second case the size of the window is specified by the width and height fields of the **Dimension** object passed in *newSize*. The dimensions are specified in pixels.

To obtain the current size of the window the method is **getSize()**. Its form is:

`Dimension getSize()`

It returns the current size of the window in contained in the width and height fields of the **Dimension** object.

A frame window created will not be visible until you call the **setVisible()** method. This method is :

`void setVisible(boolean visibleFlag)`

If *visibleFlag* is **true**, the component is visible else it is hidden.

To change the title in the frame window use the method **setTitle()**. Its form is:

`void setTitle(String newTitle)`

where *newTitle* is the new title for the window.

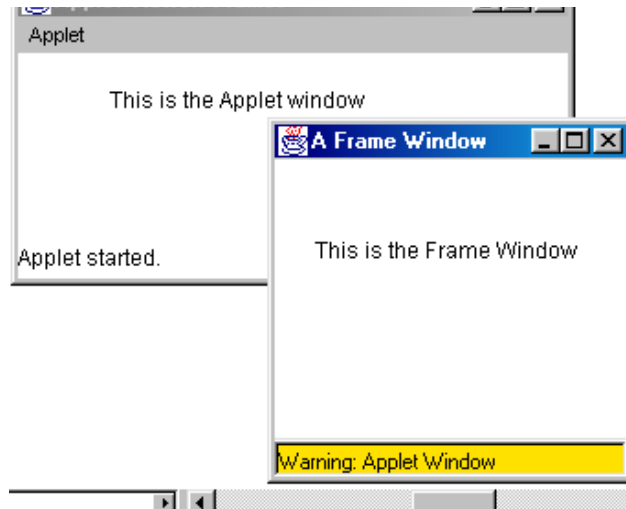
Your program must remove the window from the screen when it is closed by calling **setVisible(false)**. To intercept a window close event, you should implement the **windowClosing()** method of the **WindowListener** interface.

Inside **windowClosing()** you must remove the window from the screen.

Let us now create a **Frame** window in an Applet as shown below:

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code = "Frames" width = 300 height = 100>
</applet>
*/
class DemoFrame extends Frame {
    DemoFrame(String title) {
        super(title);
    }
    public void paint(Graphics g) {
        g.drawString("This is the Frame Window", 50, 50);
    }
    public class Frames extends Applet {
        public void init() {
            Frame f;
            f = new DemoFrame("A Frame Window");
            f.setSize(200,200);
            f.setVisible(true);
        }
        public void start() {
            f.setVisible(true);
        }
        public void stop() {
            f.setVisible(false);
        }
        public void paint(Graphics g) {
            g.drawString("This is the Applet window", 10, 30);
        }
    }
}
```

The sample output will be :



First you create a subclass of **Frame**. Once a subclass of **Frame** is created, you create an object of that class. This causes the frame window to come into existence, but you should call **setVisible()** (by setting it to true) to make it visible. You can also set the size of the window explicitly by calling **setSize()**. Next override any of the standard window methods like **init()**, **start()**, **stop()** and **paint()**. Note that we have not implemented any event handling mechanism. You can modify the above program to create an object to handle window events to close the frame window by implementing the **windowClosing()** method of the **WindowListener** interface by calling **setVisible()** .

12.3 Check Your Progress.

1. Fill in the blanks.

- The _____ method is used to set the dimensions of the Frame window.
- A frame window created will not be visible until you call the _____ method.

12.4 WORKING WITH GRAPHICS

12.4.1 Introduction

The AWT support a rich set of graphics methods. All graphics are drawn relative to a window. The window can be the main window of an applet, a child window of an applet or a stand alone application window. Each window originates at the upper left hand corner and is 0,0. All the output to a window takes place through a graphics context. A graphics context is encapsulated by the **Graphics** class and is obtained in the following ways :

- It is passed to an applet when one of its various methods like **paint()** or **update()** is called.

- It is returned by the **getGraphics()** method of **Component**.

The **Graphics** class defines a number of drawing functions. Each shape can be drawn edge only or filled. Objects are drawn and filled in the currently selected graphics color, which is black by default. Whenever a graphics object is drawn which exceeds the dimensions of the window, the output is automatically clipped. Let us see some of the drawing methods in this section.

12.4.2 Drawing Lines

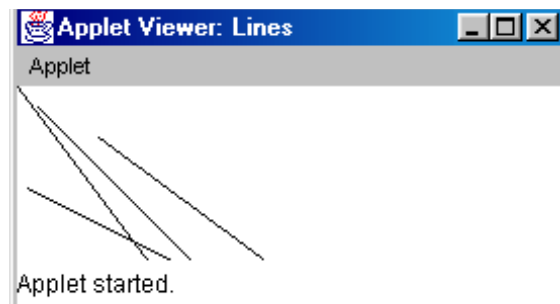
The method **drawLine()** is used to draw lines. Its form is :

```
void drawLine(int startX, int startY, int endX, int endY)
```

The line is drawn in the current drawing color beginning at *startX*, *startY* and ending at *endX*, *endY*. In the following example a number of lines are drawn :

```
import java.awt.*;
import java.applet.*;
/*
<applet code = "Lines" width = 300 height = 200>
</applet>
*/
public class Lines extends Applet {
    public void paint(Graphics g) {
        g.drawLine(10, 10, 100, 100);
        g.drawLine(0, 0, 150, 200);
        g.drawLine(5, 50, 400, 250);
        g.drawLine(40, 25, 250, 180);
    }
}
```

The output of the program would be :



12.4.3 Drawing Rectangles :

drawRect() and **fillRect()** are the methods which display an outlined and filled rectangle respectively. These methods are :

```
void drawRect(int top, int left, int width, int height)
```

```
void fillRect(int top, int left, int width, int height)
```

top, left is the upper left corner of the rectangle. *width* and *height* specify the dimensions of the rectangle.

drawRoundRect() or **fillRoundRect()** are the methods to draw rounded rectangle. A rounded rectangle has rounded corners. The methods are:

```
void drawRoundRect(int top, int left, int width, int height, int xDiam, int yDiam)
```

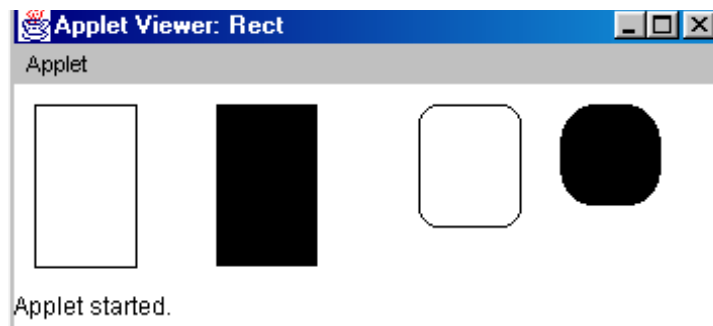
```
void fillRoundRect(int top, int left, int width, int height, int xDiam, int yDiam)
```

top, left is the upper left corner of the rectangle. *width* and *height* specify the dimensions of the rectangle. The diameter of the rounding arc along the X axis is specified by *xDiam* and the diameter of the rounding arc along the Y axis is specified by *yDiam*.

The following example illustrates

```
import java.awt.*;
import java.applet.*;
/*
<applet code = "Rect" width = 300 height = 200>
</applet>
*/
public class Rect extends Applet {
    public void paint(Graphics g) {
        g.drawRect(10, 10, 50, 80);
        g.fillRect(100, 10, 50, 80);
        g.drawRoundRect(200, 10, 50, 60, 20, 20);
        g.fillRoundRect(270, 10, 50, 50, 35, 40);
    }
}
```

output



12.4.4 Drawing Ellipses and Circles :

To draw an ellipse use the **drawOval()** method and to fill an ellipse use the **fillOval()** method. These methods are :

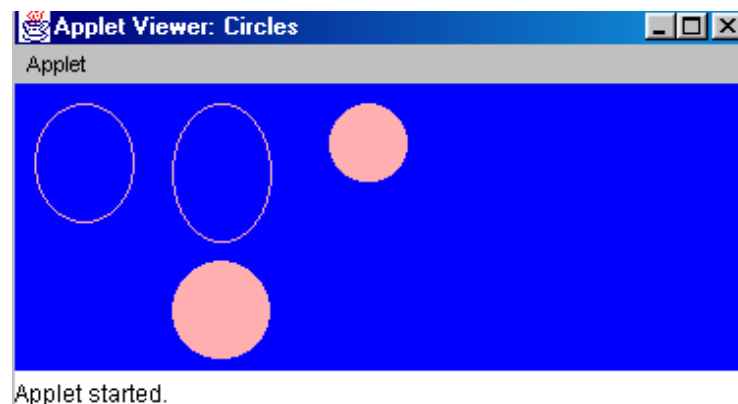
```
void drawOval(int top, int left, int width, int height)
```

```
void fillOval(int top, int left, int width, int height)
```

The ellipse is drawn within a bounding rectangle whose upper left corner is *top*, *left* and whose width and height are specified by *width* and *height* respectively. To draw a circle you will have to specify a square as the bounding rectangle. The following example draws ellipses :

```
import java.awt.*;
import java.applet.*
/*
<applet code = "Circles" width = 300 height = 200>
</applet>
*/
public class Circles extends Applets {
    public void paint(Graphics g) {
        g.drawOval(10, 10, 50, 60);
        g.drawOval(80, 10, 50, 70);
        g.fillOval(160, 10, 40, 40);
        g.fillOval(80, 90, 50, 50);
    }
}
```

output



Note : Many times you will want to size a graphics object to fit the current size of the window in which it is drawn. For this purpose, you should first obtain the current dimensions of the window by calling the **getSize()** method on the window object. It will return the dimensions of the window encapsulated within the

Dimension object. Once you obtain the size of the window, then you can scale your graphical output as per requirement.

12.4 Check Your Progress.

1. Match the following.

Column A

- a) fillOval()
- b) drawRect()
- c) fillRect()
- d) drawRoundRect()

Column B

- i) to draw a rounded rectangle
- ii) to fill an ellipse
- iii) to draw a rectangle
- iv) to fill a rectangle

12.5 COLOR

Java supports color in a portable, device independent fashion. You can specify any color that you want in the AWT. It then finds the best match for the color within the limits of the display hardware which is currently executing your program or applet. Color is encapsulated by the class **Color**. **Color** defines a number of constants to specify a number of common colors. You can also create your own colors by using the color constructors. Some of the constructors for **Color** are :

`Color(int red, int green, int blue)`

`Color(int rgbValue)`

`Color(float red, float green, float blue)`

Using the first constructor you can specify three integer values to specify the color as a mix of red, green and blue. These values must be between 0 and 255. eg.

`new Color(100, 100, 255)`

The second constructor takes a single integer value that contains a mix of red, green and blue packed in an integer. This integer is organised as : red in bits 16 to 23, green in bits 8 to 15 and blue in bits 0 to 7.

The third constructor takes three float values between 0.0 to 1.0 to specify the relative mix of red, green and blue.

The **setForeground()** and **setBackground()** are the methods which can be used to set the foreground and background colors respectively. You can also use the created color as the current drawing color.

You can obtain the red, green and blue components of a color independently using the following methods :

`int getRed()`

`int getGreen()`

`int getBlue()`

Each of these methods returns the RGB color component found in the

invoking **Color** object in the lower 8 bits of an integer.

To obtain a packed RGB representation use the method **getRGB()**. Its form is :

```
int getRGB()
```

By default graphics objects are drawn in the current foreground color. You can change this color with the method **setColor()**. This method has the following form :

```
void setColor(Color newColor)
```

where *newColor* specifies the new drawing color.

getColor() is the method used to obtain the current color. Its form is :

```
Color getColor()
```

To make use of these methods to set colors and draw various objects like lines, rectangles, circles is left as an exercise to the student.

Setting the Paint Mode

The paint mode determines how objects are drawn in a window. By default the new output to a window overwrites any pre existing contents. However, it is possible to have new objects XORed onto the window. For this purpose you use the **setXORMode()** method which is as follows :

```
void setXORMode(Color xorColor)
```

where *xorColor* specifies the color that will be XORed to the window when an object is drawn. The XOR mode guarantees that the new object is always to be visible no matter what color the object is drawn over.

To return to the overwrite mode you use the **setPaintMode()** method. Its form is :

```
void setPaintMode()
```

In general, you will want to use the overwrite mode for normal output and the XOR mode for any special purposes.

12.5 Check Your Progress.

1. Fill in the blanks.

- a) To obtain a packed RGB representation of the color we use the method_____
- b) The _____ mode determines how objects are drawn in a window.
- c) By default graphics objects are drawn in the current _____ color.

12.6 FONTS

The AWT supports multiple type fonts. Fonts have a family name, a logical font name and a face name. The family name is the general name of the font eg. Courier. The logical name specifies a category of the font eg. Monospaced. The face name specifies a specific font eg. Courier Italic. In this section let us see a

few methods of the **Font** class and use these methods in examples to set fonts, obtain font information etc. Note that only an introduction to the **Font** class is presented here. A detailed study is an area the student would like to explore further.

Fonts are encapsulated in the **Font** class which defines several methods. It also defines the following variables :

Variable	Meaning
String name	Name of the font
float pointSize	Size of the font in points
int size	Size of the font in points
int style	Font style

In this section we shall attempt an overview of the methods of the **Font** class. A detailed study of these methods may be an area the student may want to explore on his own.

To obtain information regarding the available fonts on your machine you use the **getAvailableFontFamilyNames()** method. This method is defined by the **GraphicsEnvironment** class. This method is shown below :

```
String[ ] getAvailableFontFamilyNames()
```

This method returns an array of strings that contains the names of the available font families.

The **getAllFonts()** method returns an array of **Font** objects for all the available fonts. This method is defined by the **GraphicsEnvironment** class and has the following form :

```
Font[ ] getAllFonts()
```

Both these methods are members of **GraphicsEnvironment**, so you need a **GraphicsEnvironment** reference to call them. This reference is obtained by the **getLocalGraphicsEnvironment()** static method as :

```
staticGraphicsEnvironment getLocalGraphicsEnvironment()
```

The following example illustrates how to obtain the names of the available fonts :

```
import java.applet.*;
import java.awt.*;

/*
<applet code = "Fonts" width = 400 height = 200>
</applet>
*/

public class Fonts extends Applet {
    public void paint(Graphics g) {
        String s = " ";
```

```

        String FList[ ];
        GraphicsEnvironment ge =
GraphicsEnvironment.getLocalGraphicsEnvironment();
        FList = ge.getAvailableFontFamilyNames();
        for(int i = 0; i < FList.length; i++)
            s+= FList[i] + " ";
        g.drawString(s, 10, 10);
    }
}

```

Run the program and check its output.

To select a new font, you should first construct a **Font** object that describes the font. One of the constructors of **Font** which is used for this purpose is :

```
Font(String fontName, int fontStyle, int pointSize)
```

fontName specifies the name of the desired font, which can be specified either using the logical name or the face name. The default font, if you don't explicitly set a font is the **Dialog** font. This is also the font used by your system's Dialog boxes. The style of the font is specified by *fontStyle*. It may consist of one or more of the three constants : **Font.PLAIN**, **Font.BOLD**, **Font.ITALIC**. To combine the styles you have to OR them together eg. Font.BOLD | Font.ITALIC specifies a bold, italic style. *pointSize* is used to specify the size of the font in points.

In order to use the font that you have created you should select it using the **setFont()** method which is defined by **Component**. Its general form is :

```
void setFont(Font fontObj)
```

where *fontObj* is the object that contains the desired font.

The following program illustrates the use of the methods discussed above.

```

import java.awt.*;
import java.applet.*;
/*
<applet code = "FontsDemo" width = 200 height = 200>
</applet>
*/
public class FontsDemo extends Applet {
    Font f;
    String s;
    public void init() {

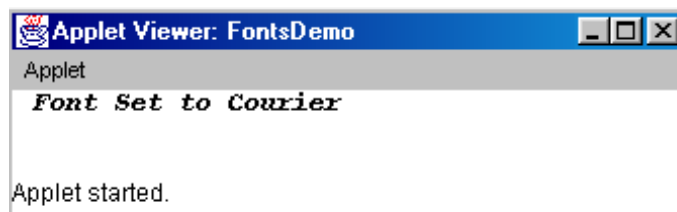
```

```

        f = new Font("Courier", Font.BOLD |
        Font.ITALIC, 14);
        setFont(f);
        s = "Font Set to Courier";
    }
    public void paint(Graphics g) {
        g.drawString(s, 10, 10);
    }
}

```

An output of the program :



To obtain information about the currently selected font, you should first get the current font by calling the method **getFont()**. This method is defined by the **Graphics** class. It has the following form :

Font getFont()

Once you have obtained the currently selected font, you can retrieve information about the font, using various methods defined by **Font** as illustrated in the example below:

```

import java.applet.*;
import java.awt.*;
/*
<applet code = "FontInfo" width = 300 height = 100>
</applet>
*/
public class FontInfo extends Applet {
    public void paint(Graphics g) {
        Font f = g.getFont();
        String name = f.getName();
        String family = f.getFamily();
        int size = f.getSize();
        int style = f.getStyle();
        String s = "Font Family : " + family;
    }
}

```

```

        s += " Font : " + name; + " Size : " + size + "
Style : ";
        if((style & Font.BOLD) == Font.BOLD)
            s+= "Bold";
        if((style & Font.ITALIC) == Font.ITALIC)
            s+= "Italic";
        if((style & Font.PLAIN) == Font.PLAIN)
            s+= "Plain";
        g.drawString(s, 20, 20);
    }
}

```

A sample output of the above program is :

12.7 SUMMARY

The AWT (Abstract Window Toolkit) contains numerous classes and methods which allow you to create and manage windows. The AWT classes are contained in the java.awt package. It is one of the largest packages of Java. The AWT defines windows according to a class hierarchy that adds functionality and specificity with each level. Those are Panel, Frame, Component, Container, Canvas.

12.8 CHECK YOUR PROGRESS - ANSWERS

12.1 & 12.2

1. a) java.awt
b) Panel, Frame
c) container
2. a) False
b) False

12.3

1. a) setSize()
b) setVisible()

12.4

1. a) - (ii)
b) - (iii)
c) - (iv)

d) - (i)

12.5

1. a) getRGB()
b) paint
c) foreground

12.6

1. a) Font
b) Dialog

12.9 QUESTIONS FOR SELF - STUDY

1. What is the use of a Frame? Describe some methods of Frame.
2. What are the ways in which you can obtain a Graphics context? Describe a few drawing methods of the Graphics class.
3. Describe the constructors of Color.
4. Describe the methods of the Font class. Also list the variables defined in the Font class.
5. **Write short notes on :**
 - a) Component Class
 - b) Panel Class
 - c) Setting the paint mode.

12.10 SUGGESTED READINGS

1. www.java.com
2. www.freejavaguide.com
3. www.java-made-easy.com
4. www.tutorialspoint.com
5. www.roseindia.net



[illegible]

AWT Controls and Layout Managers

13.0	Objectives
13.1	Introduction
13.2	AWT Controls
13.2.1	Labels
13.2.2	Buttons
13.2.3	Check Boxes
13.2.4	Checkbox Group
13.2.5	Choice Controls
13.2.6	Lists
13.2.7	Text Field
13.2.8	Scroll Bars
13.2.9	Text Area
13.3	Layout Managers
13.3.1	Fundamentals
13.3.2	Flow Layout
13.3.3	Border Layout
13.3.4	Grid Layout
13.3.5	Card Layout
13.4	Summary
13.5	Check Your Progress - Answers
13.6	Questions for Self - Study
13.7	Suggested Readings

13.0 OBJECTIVES

Dear students,

After studying this chapter you will be able to :

- explain the control components
- discuss how to use the controls like Labels, Push buttons, Check boxes, Check lists, Lists, Scroll bars, and Text editing.
- discuss the types of layouts viz. flow layout, border layout, grid layout, card layout and their use in programs.

13.1 INTRODUCTION

Controls are the components that allow the user to interact with your application. eg. a push button control. A layout manager automatically positions controls within a container. In addition to controls, a frame window can also

include a menu bar. Each entry in a menu bar activates a drop down menu of options. A user can select the option of his choice. A menu bar is always positioned at the top of a window.

The controls supported by AWT are subclasses of **Component** and include the following : Labels, Push buttons, Check boxes, Check lists, Lists, Scroll bars, and Text editing.

To include a control in the window, it has to be added to the window. For this purpose, you should create an instance of the control and add it by using the method **add()**. This method is defined by **Container**. Its form which we shall use for controls is:

```
Component add(Component compObj)
```

where *compObj* is an instance of the control that you want to add. Once a control has been added, it is automatically visible whenever its parent window is displayed.

To remove a control from the window we use the method **remove()** which is also defined by **Component**. Its general form is :

```
void remove(Component obj)
```

where *obj* is the reference to the control you want to remove. You can also remove all the controls by calling the method **removeAll()**.

Labels are passive controls whereas all the other controls generate events when they are accessed by the user. Your program therefore should implement the appropriate interface and register an event listener for each control which is to be monitored. Once you install a listener, the event will automatically be sent to it.

13.1 Check Your Progress.

1. Write True or False.

- a) Check Boxes are passive controls.
- b) Labels generate events when they are accessed by the user.
- c) A layout manager automatically positions controls within a container.

13.2 AWT CONTROLS

Let us study the various controls supported by AWT in the following sections:

13.2.1 Labels

A label is an object of type **Label** and contains a string which it displays. Labels do not support any interaction with the user and are therefore passive controls. The constructors for Label are :

```
Label()
```

```
Label(String str)
```

```
Label(String str, int how)
```

where the string is specified by *str*. This string is by default left justified. *how* specifies the alignment which will be used by *str*. The value of *how* must be one of these three constants : **Label.LEFT**, **Label.RIGHT**, **Label.CENTER**.

setText() method is used to set or change text in a label. The **getText()** method is used to obtain the current label. These methods have the following form :

```
void setText(String str)    str sets the string for the label
```

```
String getText()           returns the current label
```

To set the alignment of the string within the label you use the method :

```
void setAlignment(int how)
```

how is the alignment constant already discussed.

getAlignment() is used to obtain the current alignment :

```
int getAlignment()
```

The following example will demonstrate :

```
import java.awt.*;
```

```
import java.applet.*;
```

```
/*
```

```
<applet code = "Labels" width = 200 height = 300>
```

```
</applet>
```

```
*/
```

```
public class Labels extends Applet {
```

```
    public void init() {
```

```
        Label first = new Label("First");
```

```
        Label second = new Label("Second");    // Create Labels
```

```
        Label third = new Label("Third");
```

```
        add(first);
```

```
        add(second);                        // Add labels to applet window
```

```
        add(third);
```

```
    }
```

```
}
```

output



13.2.2 Buttons

A push button is a component that contains a label and generates an event when pressed. Push buttons are objects of type **Button**. **Button** defines the following constructors :

`Button()`

`Button(String str)`

The second constructor creates a button which contains *str* as a label.

After a button has been created, you can set its label with the **setLabel()** method. The label can be retrieved using the **getLabel()** method. These methods are :

`void setLabel(String str)`

`String getLabel()`

Each time a button is pressed an action event is generated which is sent to any listeners which have previously registered for receiving the action event notification. Each listener implements the **ActionListener** interface. This interface defines the **actionPerformed()** method. This method is called when an event occurs. It takes an **ActionEvent** object as its argument. This object contains both a reference to the button that generated the event and a reference to the string that is the label of the button. Either value may be used to identify the button.

The following example illustrates using the label of the button to determine which button is pressed. Each time a button is pressed, a message is displayed which reports which button is displayed. Note that we obtain the label by calling the **getActionCommand()** method on the **ActionEvent** object passed to **actionPerformed()**.

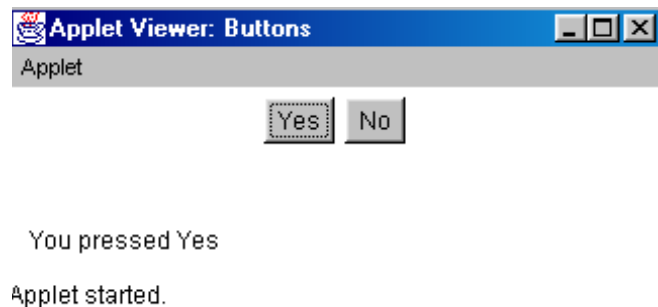
```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code = "Buttons" width = 100 height = 300>
</applet>
*/
public class Buttons extends Applet implements ActionListener {
    String s = " ";
    Button yes, no;
    public void init() {
        yes = new Button("Yes");
        no = new Button("No");
        add(yes);
        add(no);
    }
}
```

```

        yes.addActionListener(this);
        no.addActionListener(this);
    }
    public void actionPerformed(ActionEvent ae) {
        String s1 = ae.getActionCommand();
        if(s1.equals("Yes"))
            s = "You pressed Yes";
        else
            s = "You pressed No";
        repaint();
    }
    public void paint(Graphics g) {
        g.drawString(s, 10, 80);
    }
}

```

A sample output of the above program is as shown below:



The following example shows how to use the **getSource()** method to obtain the object to determine which button has been pressed. For this purpose you should keep a list of the button objects when they are created.

```

import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code = "Buttons1" width = 250 height = 100>
</applet>
*/
public class Buttons1 extends Applet implements ActionListener {
    String s = " ";
    Button bList[] = new Button[2];
}

```

```

public void init() {
    Button yes = new Button("Yes");
    Button no = new Button("No");
    bList[0] = (Button) add(yes);
    bList[1] = (Button) add(no);
    for(int i = 0; i < 2; i++)
        bList[i].addActionListener(this);
}

public void actionPerformed(ActionEvent ae) {
    for(int i = 0; i < 2; i++)
    {
        if(ae.getSource() == bList[i])
            s = "You pressed " + bList[i].getLabel();
    }
    repaint();
}

public void paint(Graphics g) {
    g.drawString(s, 10, 80);
}
}

```

Check the output of this program as an exercise.

13.2.1 & 13.2.2 Check Your Progress.

1. Fill in the blanks.

- method is used to set or change text in a label.
- Push buttons are objects of type
- The method is used to obtain the object to determine which button has been pressed.

13.2.3 Check Boxes

A check box is a control that is used to turn an option on or off. It consists of a small box that can either contain a check mark or not. There is a label associated with each check box. It describes the option that the check box represents. Check boxes can be used individually or as part of a group. Check boxes are objects of the **Checkbox** class. Its constructors are :

```

Checkbox()                      Checkbox(String str)
Checkbox(String str, boolean on)
Checkbox(String str, boolean on, CheckboxGroup cbGroup)

```

`Checkbox(String str, CheckboxGroup cbGroup, boolean on)`

str specifies the label for a check box. *on* allows you to set the initial state of the check box. If *on* is **true** then the checkbox is initially checked, otherwise it is cleared. *cbGroup* specifies the group for a checkbox. If the check box is not a part of the group then *cbGroup* must be **null**.

getState() method used to retrieve the current state of a check box. Its form is :

`boolean getState()`

setState() is used to set the state of the check box. Its form is :

`void setState(Boolean on)`

If *on* is **true**, the box is checked, if **false** the box is cleared.

To obtain the current label, you use **getLabel()** and to set the label you use **setLabel()**. These methods are as follows :

`String getLabel()` `void setLabel(String str)`

where *str* specifies the label for the check box.

Each time a check box is checked or cleared, an item event is generated which is sent to any listener which previously registered an interest in receiving item event notifications from that component. Each listener implements the **ItemListener** interface. This interface defines the **itemStateChanged()** method which takes as its argument an object of **ItemEvent**.

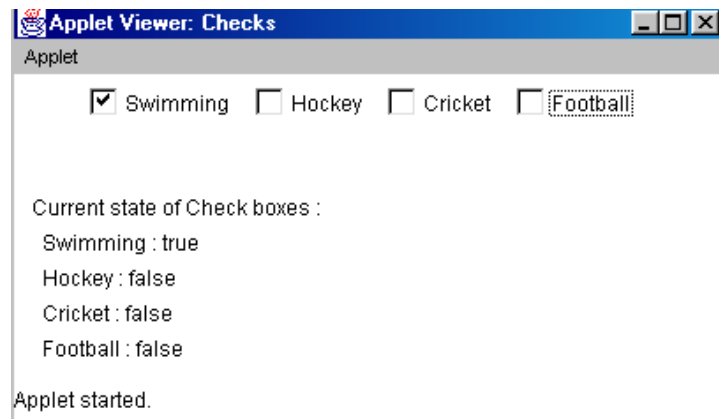
In the following example we illustrate the use of check boxes and display status of each box. Every time you change the status of the check box, the status display will be updated.

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code = "Checks" width = 240 height = 200>
</applet>
*/
public class Checks extends Applet implements ItemListener {
    String s = " ";
    Checkbox swim, hockey, cricket, football;
    public void init() {
        swim = new Checkbox("Swimming", true);
        hockey = new Checkbox("Hockey");
        cricket = new Checkbox("Cricket");
        football = new Checkbox("Football");
```

```

        add(swim);
        add(hockey);
        add(cricket);
        add(football);
        swim.addItemListener(this);
        hockey.addItemListener(this);
        cricket.addItemListener(this);
        football.addItemListener(this);
    }
    public void itemStateChanged(ItemEvent ie) {
        repaint();
    }
    public void paint(Graphics g) {
        s = "Current state of Check boxes :";
        g.drawString(s, 10, 80);
        s = " Swimming : " + swim.getState();
        g.drawString(s, 10, 100);
        s = " Hockey : " + hockey.getState();
        g.drawString(s, 10, 120);
        s = " Cricket : " + cricket.getState();
        g.drawString(s, 10, 140);
        s = " Football : " + football.getState();
        g.drawString(s, 10, 160);
    }
}
}
output

```



13.2.4 Checkbox Group

Radio buttons are a set of mutually exclusive check boxes in which one and only one check box in the group can be checked at any one time. In order to create radio buttons you must first define a group to which they belong and then specify that group when you construct the check boxes. Check box groups are objects of type **CheckboxGroup**. It has only the default constructor defined, which creates an empty group.

To determine which check box in a group is currently selected you call the **getSelectedCheckbox()** method. To set a check box you call the **setSelectedCheckbox()** method. These methods are as follows :

```
Checkbox getSelectedCheckbox()
```

```
void setSelectedCheckbox(Checkbox which)
```

where *which* is the check box you want to be selected. In this case, the previously selected check box will be turned off.

The following example will demonstrate :

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code = "CheckGroup" width = 300 height = 200>
</applet>
*/
public class CheckGroup extends Applet implements ItemListener {
    String s= " ";
    Checkbox swim, hockey, cricket, football;
    CheckboxGroup cbg;
    public void init() {
        cbg = new CheckboxGroup();
        swim = new Checkbox("Swimming", cbg, true);
        hockey = new Checkbox("Hockey", cbg, false);
        cricket = new Checkbox("Cricket", cbg, false);
        football = new Checkbox("Football", cbg, false);
        add(swim);
        add(hockey);
        add(cricket);
        add(football);
        swim.addItemListener(this);
        hockey.addItemListener(this);
```

```

        cricket.addItemListener(this);
        football.addItemListener(this);
    }
    public void itemStateChanged(ItemEvent ie) {
        repaint();
    }
    public void paint(Graphics g) {
        s = "Currently selected : ";
        s+=cbg.getSelectedCheckbox().getLabel();
        g.drawString(s, 10, 100);
    }
}

```

Run the program and check output as an exercise.

13.2.3 & 13.2.4 Check Your Progress.

1. Write True or False.

- a) A check box is a control that is used to turn an option on or off.
- b) More than one Radio buttons can be checked at a time in a checkbox group.
- c) **CheckboxGroup** has only the constructor defined, which creates an empty group.

13.2.5 Choice Controls

The **Choice** class is used to create pop up list of items from which the user can select. Thus, a *Choice control* is a form of menu. When the user clicks on a **Choice** component, the whole list of choices pops up and a new selection can be made. Each item in the list appears in the order in which it has been added to the **Choice** object. **Choice** defines only the default constructor which creates an empty list.

To add a selection to the list call **addItem()** or **add()**. The general forms of these methods are :

```
void addItem(String name)
```

```
void add(String name)
```

where *name* is the name of the item being added. To determine which item is currently selected you can call either the **getSelectedItem()** or the **getSelectedIndex()** methods. These methods are :

```
String getItem()
```

This method returns a string containing the name of the item

```
int getSelectedIndex()
```

returns the index of the item. The first item is index 0.

By default the first item added to the list is selected.

With **getItemCount()** method you can obtain the number of items in the list.

Its form is : `int getItemCount()`

You can set the currently selected item using the **select()** method with either a zero based integer index or a string that will match a name in the list. These methods are:

`void select(int index)` `void select(String name)`

You can obtain the name associated with a particular item with the **getItem()** method. Its general form is :

`String getItem(int index)`

where *index* specifies the index of the desired item.

Everytime a choice is selected, an item event is generated. This is sent to any previously registered listener. Each listener implements the **ItemListener** interface. This interface defines the **itemStateChanged()** method, which takes an **ItemEvent** object as its argument.

The following program demonstrates creating choice menus:

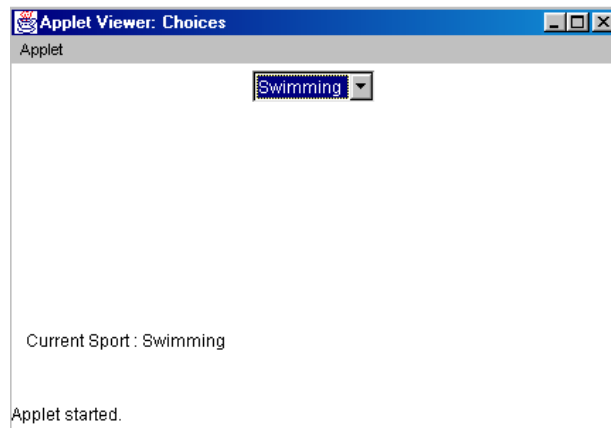
```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code = "Choices" width = 250 height = 300>
</applet>
*/
public class Choices extends Applet implements ItemListener {
    Choice sports;
    String s = " ";
    public void init() {
        sports = new Choice();
        sports.add("Swimming");
        sports.add("Hockey");
        sports.add("Cricket");
        sports.add("Football");
        add(sports);
        sports.addItemListener(this);
    }
    public void itemStateChanged(ItemEvent ie) {
```

```

        repaint();
    }
    public void paint(Graphics g) {
        s = "Current Sport : ";
        s += sports.getSelectedItem();
        g.drawString(s, 10, 200);
    }
}
}

```

output



13.2.6 Lists

The **List** class provides a compact multiple-choice scrolling selection list. In the **Choice** object, only the single selected item in the menu is displayed. However, a **List** object can be constructed to show any number of choices in the visible window. It can also be created to allow multiple selections.

Constructors for **List** are :

`List()`

`List(int numRows)`

`List(int numRows, boolean multipleSelect)`

where *numRows* specifies the number of entries in the list that will always be visible. Others are scrolled into view as needed. If *multipleSelect* is **true**, then the user may select two or more items at a time, if it is **false**, then only one item can be selected.

Use the method **add()** to add a selection to the list.

It has the following two forms :

`void add(String name)` `void add(String name, int index)`

where *name* is the name of the item to be added to the list. In the second form of the method, you can specify the index at which you wish to add the item to the list by *index*. Indexing begins at 0.

In lists which allow only single selection you can determine the currently selected item with either the **getSelectedItem()** or **getSelectedIndex()** methods.

These methods are :

```
String getItem()          int getSelectedIndex()
```

getSelectedItem() returns a string containing the name of the item. A **null** is returned if no item is selected or if more than one item is selected. The **getSelectedIndex()** method returns the index of the item. Remember that the first item is at index 0. If more than one item is selected or no selection has been done a -1 is returned.

For lists which allow multiple selection, the methods which can be used to determine the selected items are :

```
String[ ] getSelectedItems()    int[ ] getSelectedIndexes()
```

getSelectedItems() returns an array containing the names of the currently selected items. **getSelectedIndexes()** returns an array containing the indexes of the currently selected items.

The **getItemCount()** method is used to obtain the number of items in the list. Its form is :

```
int getItemCount()
```

You can set the currently selected item by using the **select()** method with a zero based integer index. Its form is :

```
void select(int index)
```

You can obtain the name associated with an item at a particular index with the **getItem()** method. Its general form is :

```
String getItem(int index)
```

where *index* specifies the index of the desired item.

Each time a **List** item is double clicked an **ActionEvent** object is generated. Its **getActionCommand()** method can be used to retrieve the name of the newly selected item. Also, everytime an item is selected or deselected with a single click, an **ItemEvent** object is generated. Its **getStateChanged()** method can be used to determine whether a selection or deselection triggered the event. **getItemSelectable()** returns a reference to the object that triggered the event.

The following example illustrates two **List** components, one with single choice and the other with multiple choice.

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code= "Lists" width = 250 height = 300>
</applet>
```

```

*/
public class Lists extends Applet implements ActionListener {
    List sports, hobbies;
    String s = " ";
    public void init() {
        sports = new List(4, true);
        hobbies = new List(5, false);
        sports.add("Swimming");
        sports.add("Hockey");
        sports.add("Cricket");
        sports.add("Football");
        hobbies.add("Reading");
        hobbies.add("Painting");
        hobbies.add("Cooking");
        hobbies.add("Drawing");
        hobbies.add("StampCollecting");
        add(sports);
        add(hobbies);
        sports.addActionListener(this);
        hobbies.addActionListener(this);
    }
    public void actionPerformed(ActionEvent ae) {
        repaint();
    }
    public void paint(Graphics g) {
        int i[ ];
        s = "Sports Selected : ";
        i = sports.getSelectedIndexes();
        for(int j = 0; j < i.length; j++)
            s+= sports.getItem(i[j]) + " ";
        g.drawString(s, 10, 200);
        s = "Hobby selected : ";
        s += hobbies.getSelectedItem();
        g.drawString(s, 10, 220);
    }
}

```

To run the program and check the results is left as an exercise to the student.

13.2.5 & 13.2.6 Check Your Progress.

1. Write True or False.

- a) A Choice control is a form of menu. .
- b) Everytime a choice is selected, an item event is generated.
- c) **List** object cannot be created to allow multiple selections.
- d) **getSelectedItem()** returns a **null** if more than one item is selected.

13.2.7 Text Field

The **TextField** class implements a single line text entry area usually called as *edit control*. Text fields allow the user to enter strings and to edit the text using arrow keys, cut and paste keys and mouse selections. **TextField** is a subclass of **TextComponent**. It defines the following constructors :

```
TextField()           TextField(int numChars)
TextField(String str)  TextField(String str, int numChars)
```

The first form creates a default text field. The second form creates a text field which is *numChars* wide. The third form initialises the text field with the string contained in *str*, whereas in the fourth form you can initialise the text field and set its width.

To obtain the string currently contained in the text field you call the **getText()** method and to set the text you call the **setText()** method. These methods are :

```
String getText()
void setText(String str)
where str is the new string.
```

You can select a portion of the text in the text field and you can also select a portion of the text under program control. Both these can be achieved with the **select()** method. The currently selected text can be obtained by your program by using **getSelectedText()**. These methods are :

```
String getSelectedText()
void select(int startIndex, int endIndex)
```

The **select()** method selects characters beginning at *startIndex* and ending at *endIndex*-1.

To control whether the contents of the text field may be modified by the user or not you use the **setEditable()** method. Its form is :

```
void setEditable(boolean canEdit)
```

Here, if *canEdit* is **true** then the text maybe changed, if it is **false** the text cannot be altered.

isEditable() is the method used to determine the editability of the text field. Its form is :

```
boolean isEditable()
```

it returns **true** if the text maybe changed, **false** otherwise.

You can disable the character echoing on the screen as they are typed by the **setEchoChar()** method. This method specifies a single character that the text field will display when characters are entered. This means that the actual characters typed will not be shown. This is particularly useful when entering passwords.

This method has the form :

```
void setEchoChar(char ch)
```

where *ch* specifies the character to be echoed.

You can retrieve the echo character by calling the **getEchoChar()** method.

Its form is :

```
char getEchoChar()
```

echoCharIsSet() is the method which can be used to check whether a text field is in this mode. Its form is :

```
boolean getCharIsSet()
```

Text fields perform their own editing functions. So your program do not need to respond to individual key events that occur within the text field. However, when the user presses Enter, you may want to respond. This generates an action event.

Let us create the user name and password screen in the following example which will make use of many methods which have been discussed above :

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code = "TextFieldDemo" width = 300 height = 200>
</applet>
*/
public class TextFieldDemo extends Applet implements ActionListener {
    TextField name, pass;
    public void init() {
        Label n = new Label("Name : ", Label.RIGHT);
        Label p = new Label("Password : ", Label.RIGHT);
        name = new TextField(15);
        pass = new TextField(5);
        pass.setEchoChar('*');

        add(n);
```



```

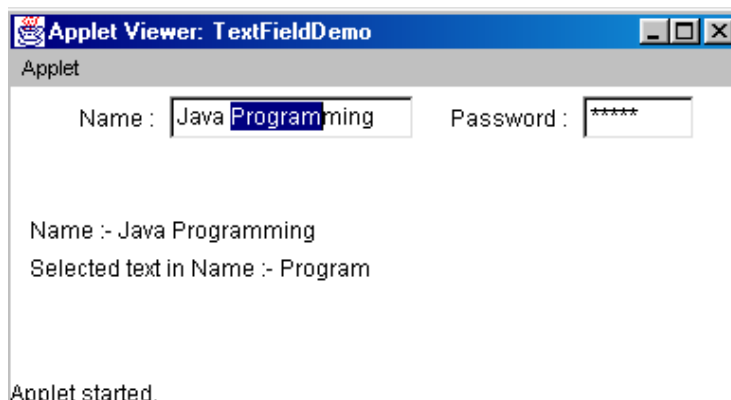
        add(name);
        add(p);
        add(pass);
        name.addActionListener(this);
        pass.addActionListener(this);
    }

    public void actionPerformed(ActionEvent ae) {
        repaint();
    }

    public void paint(Graphics g) {
        g.drawString("Name :- " + name.getText(), 10, 80);
        g.drawString("Selected text in Name :- " +
            name.getSelectedText(), 10, 100);
    }
}

```

A sample output of the program is as shown below :



13.2.8 Scroll Bars

Scroll bars are used to select continuous values between a specified minimum and maximum. Scroll bars can be oriented horizontally or vertically. A scroll bar is in fact a composite of a number of individual parts. Each end of the scroll bar has an arrow. You can click on these arrows to move the current value of the scroll bar by one unit in the direction of the arrow. The current value of the scroll bar relative to its minimum and maximum values is indicated by the slider box for the scroll bar. The slider box (or *thumb*) can be dragged by the user to a new position. The scroll bar then reflects this value. If the user clicks in the background space on either side of the thumb, then it causes the thumb to jump in that direction by an increment value larger than 1. Typically, this action

translates into some form of page up or page down. The **Scrollbar** class encapsulates scroll bars. **Scrollbar** defines the following constructors :

```
Scrollbar()                               Scrollbar(int style)
Scrollbar(int style, int initialValue, int thumbSize, int min, int max)
```

The first form creates a vertical scroll bar. With the second and third forms you can specify the orientation of the scroll bar. If style is **Scrollbar.VERTICAL** a vertical scroll bar is created. If style is **Scrollbar.HORIZONTAL** then a horizontal scrollbar is created. The third form allows you to pass the initial value of the scroll bar in *initialValue*. The number of units represented by the height of the thumb is passed in *thumbSize*. *min* and *max* specify the minimum and maximum values of the scroll bar respectively.

The **setValues()** method is used to set the parameters for the scroll bar if the first or the second form of constructor is used for constructing the scroll bar. The **setValues()** method has the form:

```
void setValues(int initialValue, int thumbSize, int min, int max)
```

Here, the parameters have the same meaning as they have in the third form of constructor described above.

getValue() is invoked to obtain the current value of the scroll bar. It returns the current setting. **setValue()** is called to set the current value. These methods are :

```
int getValue()
void setValue(int newValue)
```

newValue specifies the new value for the scroll bar. When you set value the slider box inside the scroll bar will be positioned to reflect the new value.

The minimum and maximum values can also be retrieved using the **getMinimum()** and **getMaximum()** methods which are:

```
int getMinimum()
int getMaximum()
```

They both return the requested value.

Each time a scroll bar is scrolled up or down one line, then by default 1 is the increment added to or subtracted from the scroll bar. This increment can be changed by calling the **setUnitIncrement()** method. page up and page down increments are 10 by default. This value can also be changed by calling the **setBlockIncrement()** method. These methods are :

```
void setUnitIncrement(int newIncr)
void setBlockIncrement(int newIncr)
```

The **AdjustmentListener** interface is needed to process scroll bar events. Every time the user interacts with a scroll bar, an **AdjustmentEvent** object is generated. The **getAdjustmentType()** method is used to determine the type of the adjustment. Following are the adjustment types :

BLOCK_DECREMENT	A page-down event has been generated
BLOCK_INCREMENT	A page-up event has been generated
TRACK	An absolute tracking event has been generated
UNIT_DECREMENT	The line-down button in a scroll bar has been pressed
UNIT_INCREMENT	The line-up button in a scroll bar has been pressed

In the following example, we shall create both horizontal and vertical scroll bars and display the current settings of the scroll bar. The scroll bars will be updated if you drag the mouse while inside the window.

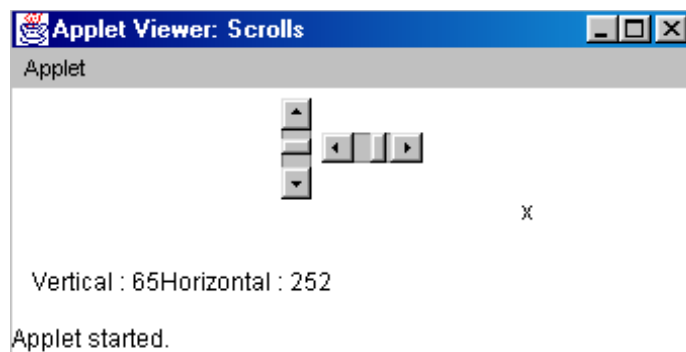
```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code = "Scrolls" width = 300 height = 150>
</applet>
*/
public class Scrolls extends Applet
    implements AdjustmentListener, MouseMotionListener {
    String s = " ";
    Scrollbar vert, hor;
    public void init() {
        int width =
Integer.parseInt(getParameter("width"));
        int height =
Integer.parseInt(getParameter("height"));
        vert = new Scrollbar(Scrollbar.VERTICAL, 0, 1,
0, height);
        hor = new Scrollbar(Scrollbar.HORIZONTAL, 0,
1, 0, width);
        add(vert);
        add(hor);
        vert.addAdjustmentListener(this);
        hor.addAdjustmentListener(this);
    }
}
```

```

        addMouseMotionListener(this);
    }
    public void adjustmentValueChanged(AdjustmentEvent
ae) {
        repaint();
    }
    public void mouseDragged(MouseEvent me) {
        int x = me.getX();
        int y = me.getY();
        vert.setValue(y);
        hor.setValue(x);
        repaint();
    }
    public void mouseMoved(MouseEvent me) {
    } // implement mouseMoved
    public void paint(Graphics g) {
        s = "Vertical : "+ vert.getValue();
        s += "Horizontal : " + hor.getValue();
        g.drawString(s, 10, 100);
        g.drawString("x", hor.getValue(), vert.getValue());
        // current mouse drag position
    }
}

```

A sample output of the above program :



13.2.7 & 13.2.8 Check Your Progress.

1. Fill in the blanks.

- a) **TextField** class implements a single line text entry area called as
- b) **isEditable()** returnsif the text in a textfield maybe changed.
- c) are used to select continuous values between a specified minimum and maximum.
- d) Each time a scroll bar is scrolled up one line, by default is the increment added to the scroll bar.

13.2.9 Text Area

The AWT includes a simple multiline editor called **TextArea**. Its constructors are :

`TextArea()` `TextArea(int numLines, int numChars)`

`TextArea(String str)` `TextArea(String str, int numLines, int numChars)`

`TextArea(String str, int numLines, int numChars, int sBars)`

numLines specifies the height, in terms of lines of the text area and *numChars* specifies its width, in characters. Initial text can be specified in *str*. *sBars* allows to specify the scroll bars you want the control to have. It can have one of **SCROLLBARS_BOTH** **SCROLLBARS_NONE** the **SCROLLBARS_HORIZONTAL_ONLY** **SCROLLBARS_VERTICAL_ONLY** following values:

TextArea is a subclass of **TextComponent**. It supports the **getText()**, **setText()**, **getSelectedText()**, **select()**, **isEditable()**, and **setEditable()** methods which have been previously described.

TextArea adds the following methods :

`void append(String str)` : appends the string specified by *str* to the end of the current text.

`void insert(String str, int index)` : inserts the string specified by *str* at the specified index.

`void replaceRange(String str, int startIndex, int endIndex)` : This method replaces characters from *startIndex* to *endIndex*-1. The replacement text is passed in *str*.

Text areas are almost self contained controls.

The following program demonstrates the use of text areas :

```
import java.awt.*;
```

```

import java.applet.*;
/*
<applet code = "TextDemo" width = 400 height = 250>
</applet>
*/
public class TextDemo extends Applet {
    public void init() {
        String val = "Scroll bars are used to select
continuous values\n" +
            " between a specified minimum
and maximum.\n" +
            " Scroll bars can be oriented
horizontally or vertically.\n" +
            "A scroll bar is in fact a composite
of\n" +
            " a number of individual parts.\n ";
        TextArea text = new TextArea(val, 10, 30);
        add(text);
    }
}

```

Check output of the above program to study the application of **TextArea** control.

13.2.9 Check Your Progress.

1. Fill in the blanks.

- TextArea** is a editor.
- TextArea** is a subclass of

13.3 LAYOUT MANAGERS

13.3.1 Fundamentals

In all our previous examples, all the components have been positioned by the default layout manager. A layout manager automatically arranges the controls within a window. Each **Container** object has a layout manager associated with it. A layout manager is an instance of any class that implements the **LayoutManager** interface. The layout manager is set by the **setLayout()** method. If no call to **setLayout()** is made, then the default layout manager is used. Whenever a container is resized (or sized for the first time), the layout manager is used to position the components in it.

The **setLayout()** method has the following form :

```
void setLayout(LayoutManager layoutObj)
```

where *layoutObj* is a reference to the desired layout manager. It is also possible to disable the layout manager and position the components manually. For this purpose, you pass a **null** for *layoutObj*. In such a situation, you should determine the shape and position of each component manually by using the **setBounds()** method defined by **Component**. Normally layout manager is used.

Each layout manager keeps a track of the list of components which are stored by their names. The layout manager is notified everytime you add a component to a container. Each component that is being managed by the layout manager contains the **getPreferredSize()** and **getMinimumSize()** methods. These return the preferred size and the minimum size required for each component. Whenever the container needs to be resized, the **minimumLayoutSize()** and **preferredLayoutSize()** methods are used. In this section let us study the predefined **LayoutManager** classes provided by Java.

13.3.2 Flow Layout

This is the default layout manager. This was the layout manager which was used by all our preceding examples. In the **FlowLayout**, components are laid out from the upper left hand corner, left to right and top to bottom. When no more components fit on a line, the next component appears on the next line. A small space is left between each component, above and below it as well as to the left and right.

The constructors for **FlowLayout** :

```
FlowLayout()
```

```
FlowLayout(int how)      FlowLayout(int how, int horz, int vert)
```

In the first form, all components are centered and a space of 5 pixels is left between each component. In the second form, you can specify how each line is aligned. The valid values for *how* are :

FlowLayout.LEFT FlowLayout.CENTER FlowLayout.RIGHT

These values specify the left, center and right alignment respectively. The third form allows to specify the horizontal and vertical space left between components in *horz* and *vert* respectively.

Let us modify our previous example of **CheckBox** control so that it uses the right aligned flow layout. Note that this program is for the purpose of illustration of the layout manager. No event handling mechanisms are incorporated.

```
import java.awt.*;
import java.applet.*;
/*
<applet code = "CheckGroup" width = 300 height = 200>
</applet>
```

```

*/
public class CheckGroup extends Applet {
    Checkbox swim, hockey, cricket, football;
    public void init() {
        setLayout(new FlowLayout(FlowLayout.RIGHT));
        swim = new Checkbox("Swimming", null, true);
        hockey = new Checkbox("Hockey");
        cricket = new Checkbox("Cricket");
        football = new Checkbox("Football");
        add(swim);
        add(hockey);
        add(cricket);
        add(football);
    }
}

```

Study the output of the above program as an exercise.

13.3.3 Border Layout :

The **BorderLayout** class implements a common layout style for top level windows. It has four narrow, fixed width components at the edges and one large area in the center. The four sides are referred to as north, south, east and west. The middle area is called the center. The constructors for **BorderLayout** are :

```

BorderLayout()           BorderLayout(int horz, int vert)

```

The first form creates the default border layout. In the second form you can specify the horizontal and vertical space left between the components by *horz* and *vert* respectively.

BorderLayout defines the following constants which specify the regions :

```

BorderLayout.CENTER           BorderLayout.SOUTH
BorderLayout.EAST             BorderLayout.WEST
BorderLayout.NORTH

```

These constants are to be used while adding the components with the following form of **add()** which is defined by **Container** :

```

void add(Component compObj, Object region);

```

compObj is the component to be added and *region* specifies where the component will be added.

The following example demonstrates the **BorderLayout** :

```

import java.awt.*;
import java.applet.*;

```



```

import java.util.*;

/*
<applet code = "BorderDemo" width = 300 height = 200>
</applet>
*/

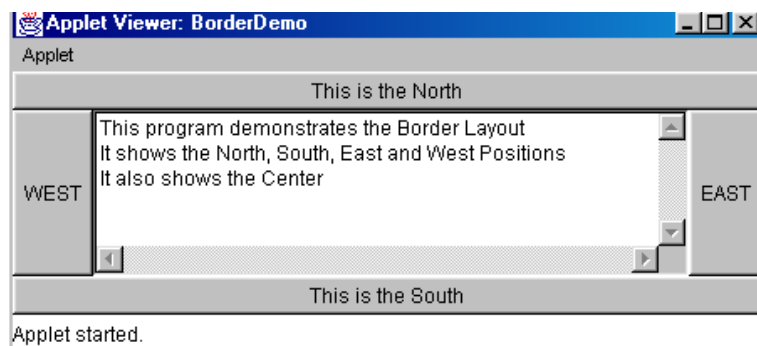
public class BorderDemo extends Applet {
    public void init() {
        setLayout(new BorderLayout());
        add(new Button("This is the North"),
BorderLayout.NORTH);
        add(new Button("This is the South"),
BorderLayout.SOUTH);
        add(new Button("EAST"), BorderLayout.EAST);
        add(new Button("WEST"), BorderLayout.WEST);
        String s = "This program demonstrates the
Border Layout \n" +

                "It shows the North, South, East
and West Positions \n" +

                "It also shows the Center";
        add(new TextArea(s),
BorderLayout.CENTER);
    }
}

```

The output of the above program is :



13.3.1 to 13.3.3 Check Your Progress.

1. Write True or False.

- Each **Container** object has a layout manager associated with it.
- BorderLayout** is the default Layout Manager.

13.3.4 Grid Layout :

This layout lays out the components in a two dimensional grid. You define the number of rows and columns when you instantiate the **GridLayout**. The constructors for **GridLayout** are :

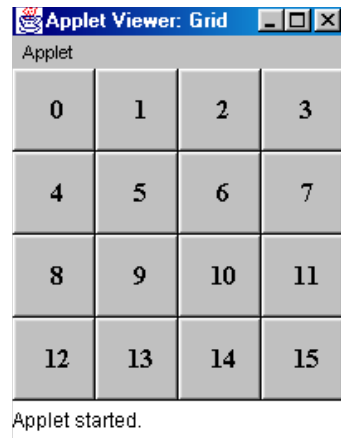
```
GridLayout()    GridLayout(int numRows, int numColumns)
GridLayout(int numRows, int numColumns, int horz, int vert)
```

The first form creates a single column grid layout. With the second form you can create a grid layout of the specified number of rows and columns. The third form allows you to specify the horizontal and vertical space between components using the *horz* and *vert* respectively. Either *numRows* or *numCols* can be zero. If *numRows* is zero, it allows columns of unlimited length and specifying *numColumns* as zero allow for unlimited-length rows.

Study the following example that creates a grid layout of size 4 x 4 and fills it with buttons, where the button of each label is its index.

```
import java.awt.*;
import java.applet.*;
/*
<applet code = "Grid" width = 200 height = 200>
</applet>
*/
public class Grid extends Applet {
    static final int n = 4;
    public void init() {
        setLayout(new GridLayout(n, n));
        setFont(new Font("Courier", Font.BOLD, 16));
        for(int i = 0; i < n; i++) {
            for(int j = 0; j < n; j++) {
                int k = i*n + j;
                add(new Button(" " + k));
            }
        }
    }
}
```

The output of the program will be :



13.3.5 Card Layout :

The **CardLayout** stores several different layouts. Each layout can be thought of as being on a separate index card in a deck that can be shuffled so that any card is on top at a given time.

CardLayout provides the following constructors :

`CardLayout()` `CardLayout(int horz, int vert)`

In the first form, you create the default card layout. With the second form you can specify the horizontal and vertical spaces left between components by *horz* and *vert* respectively.

The cards are typically held in an object of type **Panel**. This panel must have a **CardLayout** as its layout manager. The cards which form the deck are also typically objects of type **Panel**. This means that you create a panel that contains the deck and a panel for each of the cards in the deck. Then, you must add the required components for each card. All the panels thus created must then be added to the panel for which the **CardLayout** is the layout manager. Finally, this panel is added to the main applet panel. After this is complete, you must allow the user some means to select between these cards. One common way is to provide a push button for each card in the deck.

You make use of **add()** to add cards to a panel.

The following form of **add()** is used :

`void add(Component panelObj, Object name);`

where *name* is the string that specifies the name of the card whose panel is specified by *panelObj*.

Once you have created a deck, a card is activated by your program by calling one of the following methods defined by **CardLayout** :

`void first(Container deck)` `void last(Container deck)`
`void next(Container deck)` `void previous(Container deck)`
`void show(Container deck, String cardName)`

where *deck* is a reference to the container (usually a panel) which holds the

cards, and *cardName* is the name of the card. **first()** causes the first card in the deck to be shown and **last()** causes the last card to be shown. To show the next card call **next()**. To show the previous card call **previous()**. The **show()** method displays the card whose name is passed in *cardName*.

The following example illustrates a two level card deck which allows the user to select either sports or colors each of which is displayed in a card. Each card is activated by pushing its button. By clicking the mouse you can cycle through the cards.

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code = "Cards" width = 300 height = 100>
</applet>
*/
public void Cards extends Applet implements ActionListener, MouseListener
{
    Checkbox cricket, hockey, football, red, blue, green;
    Panel SportsColors;
    CardLayout clo;
    Button Sports, Colors;
    public void init() {
        Sports = new Button("Sports");
        Colors = new Button("Colors");
        add(Sports);
        add(Colors);
        clo = new CardLayout();
        SportsColors = new Panel();
        SportsColors.setLayout(clo);
        cricket = new Checkbox("Cricket", null, true);
        hockey = new Checkbox("Hockey");
        football = new Checkbox("Football");
        red = new Checkbox("Red", null, true);
        blue = new Checkbox("Blue");
        green = new Checkbox("Green");
        Panel sPanel = new Panel();
        sPanel.add(cricket);
        sPanel.add(hockey);
```

```

        sPanel.add(football);
        Panel cPanel = new Panel();
        cPanel.add(red);
        cPanel.add(blue);
        cPanel.add(green);
        SportsColors.add(sPanel, "Sports");
        SportsColors.add(cPanel, "Colors");
        add(SportsColors);
        Sports.addActionListener(this);
        Colors.addActionListener(this);
        addMouseListener(this);
    }

    public void mousePressed(MouseEvent me) {
        clo.next(SportsColors);    // Cycle through cards
    }

    // empty implementations for other
    MouseListener methods

    public void mouseClicked(MouseEvent me) {
    }

    public void mouseEntered(MouseEvent me) {
    }

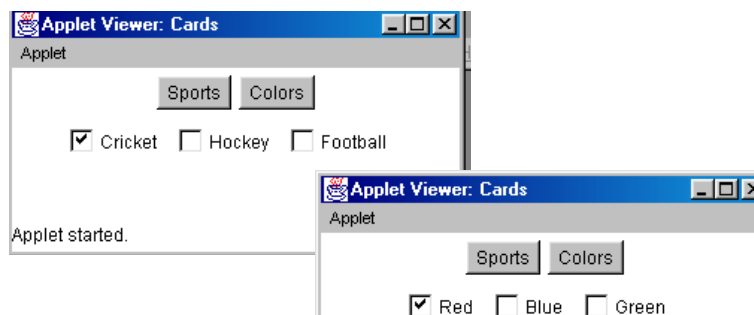
    public void mouseExited(MouseEvent me) {
    }

    public void mouseReleased(MouseEvent me) {
    }

    public void actionPerformed(ActionEvent ae) {
        if(ae.getSource() == Sports)
            clo.show(SportsColors, "Sports");
        else
            clo.show(SportsColors, "Colors");
    }
}
}

```

output



13.3.4 to 13.3.5 Check Your Progress.

1. Answer the following.

- a) Write down the constructors for a **GridLayout**.

- b) Write the methods defined by **CardLayout** to activate a card.

13.4 SUMMARY

Controls are the components that allow the user to interact with your application. eg. a push button control. A layout manager automatically positions controls within a container. In addition to controls, a frame window can also include a menu bar. Each entry in a menu bar activates a drop down menu of options. A user can select the option of his choice. A menu bar is always positioned at the top of a window.

The controls supported by AWT are subclasses of Component and include the following : Labels, Push buttons, Check boxes, Check lists, Lists, Scroll bars, and Text editing.

Source : *books.google.co.in* (Google book)

13.5 CHECK YOUR PROGRESS - ANSWERS

13.1

1. a) False
b) False
c) True

13.2.1 & 13.2.2

1. a) setText
b) Button
c) getSource()

13.2.3 & 13.2.4

1. a) True

- b) False
- c) True

13.2.5 & 13.2.6

1. a) True
 - b) True
 - c) False
 - d) True

13.2.7 & 13.2.8

1. a) edit control
 - b) true
 - c) Scrollbars
 - d) 1

13.2.9

1. a) multiline
 - b) TextComponent

13.3.1 to 13.3.3

1. a) True
 - b) False

13.3.4 & 13.3.5

1. a) The constructors for **GridLayout** are: `GridLayout()`, `GridLayout(int numRows, int numColumns)`,
`GridLayout(int numRows, int numColumns, int horz, int vert)`
 - b) The methods defined by **CardLayout** to activate a card are :
`void first(Container deck)`, `void last(Container deck)`, `void next(Container deck)`, `void previous(Container deck)`, `void show(Container deck, String cardName)`

13.6 QUESTIONS FOR SELF - STUDY

1. What do you understand by controls? List the controls supported by AWT. Explain how to include a control in a window.
2. Describe in detail any 5 controls supported by the AWT.
3. Describe the various methods of the **TextField** class.

4. **Write short notes on :**
 - a) Scroll Bars
 - b) TextArea
 - c) GridLayout
 - d) CardLayout
5. Describe the **FlowLayout** and **Border Layout** in detail.

13.7 SUGGESTED READINGS

1. www.java.com
2. www.freejavaguide.com
3. www.java-made-easy.com
4. www.tutorialspoint.com
5. www.roseindia.net



[illegible]

[illegible]

Chapter : 14

Introduction to Swing

14.0	Objectives
14.1	Introduction
14.2	Icons and Labels
14.2.1	Icon
14.2.2	Lables
14.3	Text Fields
14.4	Buttons
14.5	CheckBoxes
14.6	Radio Buttons
14.7	Combo Boxes
14.8	Tabbed Panes
14.9	Scroll Panes
14.10	Trees
14.11	Tables
14.12	Summary
14.13	Check Your Progress - Answers
14.14	Questions for Self - Study
14.15	Suggested Readings

14.0 OBJECTIVES

Dear students,

After studying this chapter you will able to :

- discuss Swing components like buttons, checkboxes and labels
- discuss components like tabbed panes, trees and tables

14.1 INTRODUCTION

Swing is a set of classes which provide more flexible and powerful components than those that are possible in the AWT. Swing provides the components such as buttons, check boxes and labels, but additionally it also provides components like tabbed panes, scroll panes, trees and tables. Here we present only a brief overview of **Swing**.

We know that the AWT components are implemented by platform specific code. Swing components on the other hand, are written entirely in Java and therefore are platform independant. The term *lightweight* is used to describe such elements. Fundamental to **Swing** is the **JApplet** class which extends **Applet**. **JApplet** is rich with functionality that is not provided by **Applet**. Remember

however, that when adding a component to an instance of **JApplet**, do not invoke the **add()** method of the applet. Instead call **add()** of the content pane of the **JApplet** object. The content pane is obtained by the following method :

```
Container getContentPane()
```

To add a component we use the method **add()** of **Container** as :

```
void add(comp)
```

where *comp* is the component to be added.

14.2 ICONS AND LABELS

14.2.1 Icons

In **Swing**, icons are encapsulated by the **ImageIcon** class, which paints an icon from an image. Its constructors are :

```
ImageIcon(String filename)           ImageIcon(URL url)
```

The first form of the constructor uses the image in the file named *filename*. The second form uses the image in the resource identified by *url*. **ImageIcon** class implements the **Icon** interface which declares the following methods :

Method	Description
int getIconHeight()	returns the height of the icon in pixels
int getIconWidth()	returns the width of the icon in pixels
void paintIcon(Component <i>comp</i> , Graphics <i>g</i> , int <i>x</i> , int <i>y</i>)	paints the icon at position <i>x</i> , <i>y</i> on the graphics context <i>g</i> .

14.2.2 Labels

Swing labels are instances of the **JLabel** class which extends **JComponent**. It can display text and/or icon. Some of its constructors are :

```
JLabel(Icon i)                      JLabel(String s)
JLabel(String s, Icon i, int align)
```

where *s* and *i* are the text and icon used for the label. The **align** argument is either **LEFT**, **RIGHT** or **CENTER**. These constants are defined in the **SwingConstants** interface.

The methods with which the icons and the text associated with the label can be read and written are :

```
Icon getIcon()                      String getText()
void setIcon(Icon i)              void setText(String s)
```

where *i* and *s* are icon and text respectively.

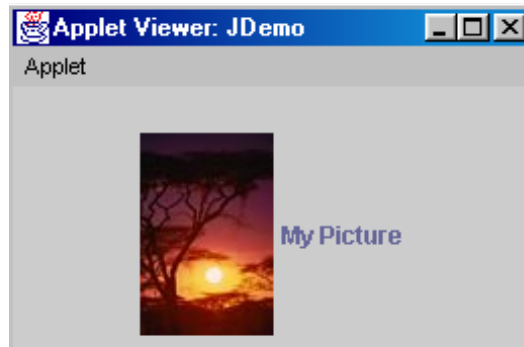
Let us see how to display text and icon with the following example

```
:
import java.awt.*;
import javax.swing.*;
```

```

/*
<applet code = "JDemo" width = 300 height = 60>
</applet>
*/
public class JDemo extends JApplet {
    public void init() {
        Container cp = getContentPane();    // get content
        pane
        ImageIcon ii = new ImageIcon("pic1.gif");
        // create an icon
        JLabel jl = new JLabel("My Picture, ii, JLabel.CENTER); // create
label
        cp.add(jl);    // Add label to content pane
    }
}
output

```



14.1 & 14.2 Check Your Progress.

1. Write True or False.

- Swing components are lightweight.
- Icons are instances of the JLabel class.
- Applet class extends JApplet.

14.3 TEXT FIELD

JTextComponent encapsulates the Swing text field and extends **JComponent**. One of its subclasses is the **JTextField** which allows you to edit a single line of text. Some of its constructors are :

```

JTextField()                      JTextField(int cols)
JTextField(String s, int cols)    JTextField(String s)

```

s is the string to be presented and *cols* is the number of columns in the text field.

The following example illustrates :

```
import java.awt.*;
import javax.swing.*;

/*
<applet code = "JTextFieldDemo" width = 300 height = 50>
</applet>
*/

public class JTextFieldDemo extends JApplet {
    JTextField jtf;
    public void init() {
        Container cp = getContentPane();
        cp.setLayout(new FlowLayout());           // set flow layout
        jtf = new JTextField(15);                // set text field
        cp.add(jtf);                             // add to content pane
    }
}
```

Run the program and study how to obtain the text field.

14.4 BUTTONS

Swing buttons are subclasses of the **AbstractButton** class, which extends **JComponent**. This class contains many methods which allow you to control the behaviour of buttons, checkboxes, and radio buttons. It is possible to associate an icon with a swing button. You can define different icons that are displayed for a component when it is disabled, pressed or selected. Another icon can be used as a rollover icon which is displayed when the mouse is positioned over that component. The methods which control this behaviour are:

```
void setDisabledIcon(Icon di)   void setPressedIcon(Icon pi)
void setSelectedIcon(Icon si)   void setRolloverIcon(Icon ri)
```

where *di*, *pi*, *si* and *ri* are the icons to be used for these different conditions. The text associated with the button can be read and written with the following methods :

```
String getText()
void setText(String s)
```

respectively, where *s* is the text to be associated with the button.

Concrete subclasses of **AbstractButton** generate action events when they are pressed. The methods to register and unregister for these events are :

```
void addActionListener(ActionListener al)
void removeActionListener(ActionListener al)
where al is the action listener.
```

AbstractButton is the superclass for push buttons, check boxes and radio buttons. Let us see each of these in the following sections :

The JButton Class : This class allows an icon, a string or both to be associated with a push button. Some of its constructors are :

```
JButton(Icon i)                JButton(String s)
JButton(String s, Icon i)
```

where *s* and *i* are string and icon used for the button respectively.

The following program demonstrates use of buttons. It displays three pushbuttons and a text field. Each pushbutton displays a menu item. The applet is created by getting the content pane and setting the layout manager for the pane. Image buttons for menu items are created and added to the pane. The applet is then registered to receive action events that are generated by the buttons. A text field is also created and added to the applet. The handler for action events displays the command string, which is associated with the button in the text field.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

/*
<applet code = "JButtons" width = 300 height = 50>
</applet>
*/

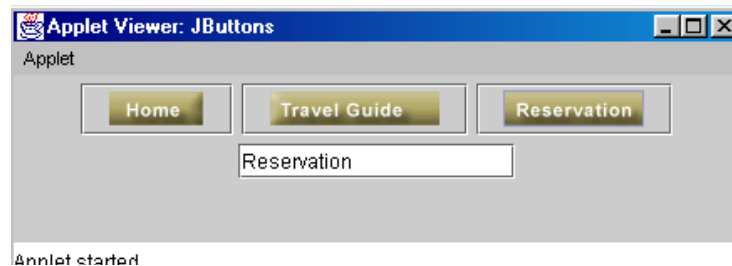
public class JButtons extends JApplet implements ActionListener {
    JTextField jtf;
    public void init() {
        Container cp = getContentPane();
        cp.setLayout(new FlowLayout());
        ImageIcon home = new
        ImageIcon("Home.gif");
        JButton jb = new JButton(home);
        jb.setActionCommand("Home");
        jb.addActionListener(this);
        cp.add(jb);
        ImageIcon travel = new
        ImageIcon("Travel.gif");
        JButton jb1 = new JButton(travel);
```

```

        jb1.setActionCommand("Travel");
        jb1.addActionListener(this);
        cp.add(jb1);
        ImageIcon reserve = new
        ImageIcon("Reservation.gif");
        JButton jb2 = new JButton(reserve);
        jb2.setActionCommand("Reservation");
        jb2.addActionListener(this);
        cp.add(jb2);
        jtf = new JTextField(15);
        cp.add(jtf);
    }
    public void actionPerformed(ActionEvent ae) {
        jtf.setText(ae.getActionCommand());
    }
}

```

output



14.3 & 14.4 Check Your Progress.

1. Fill in the blanks.

- allows you to edit a single line of text in Swing.
- is the superclass for push buttons, check boxes and radio buttons.
- A icon is displayed when the mouse is positioned over a component.

14.5 CHECKBOXES

The **JCheckBox** class provides the functionality of the check box and it is a concrete implementation of **AbstractButton**. Some of its constructors are :

`JCheckBox(Icon i)`

`JCheckBox(Icon i, boolean state)`

`JCheckBox(String s)`

`JCheckBox(String s, boolean state)`

`JCheckBox(String s, Icon i)` `JCheckBox(String s, Icon i, boolean state)`

where *i* is the icon for the button, *s* specifies the text. If state is **true**, then the checkbox is initially selected, otherwise it is not. The state of the checkbox can be changed with the following method :

`void setSelected(boolean state)`

state is **true** if the check box should be checked.

Let us create an applet to display check boxes and a text field. When a checkbox is pressed, its text will be displayed in the text field. Obtain the content pane for the **JApplet** object and set the layout. Then add check boxes to the content pane. Register the applet to receive the item events. Finally add a text field to the content pane. Whenever a check box is selected or deselected an item event is generated. This is handled by the **itemStateChanged()** method. Inside **itemStateChanged()**, the **getItem()** method will get the **JCheckBox** object which generated the event. Then the **getText()** method gets the text for that check box and sets the text inside the text field.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
/*
    <applet code = "JCheckBoxes" width = 350 height = 40>
        </applet>
*/
public class JCheckBoxes extends JApplet implements ItemListener {
    JTextField jtf;
    public void init() {
        Container cp = getContentPane();
        cp.setLayout(new FlowLayout());
        JCheckBox cb = new
        JCheckBox("Swimming");
        cb.addItemListener(this);
        cp.add(cb);
        JCheckBox cb1 = new
        JCheckBox("Skating", true);
        cb1.addItemListener(this);
        cp.add(cb1);
        JCheckBox cb2 = new
        JCheckBox("Riding");
        cb2.addItemListener(this);
        cp.add(cb2);
    }
}
```

```

        jtf = new JTextField(15);
        cp.add(jtf);
    }
    public void itemStateChanged(ItemEvent ie) {
        JCheckBox cb = (JCheckBox)ie.getItem();
        jtf.setText(cb.getText());
    }
}

```

Study the output of the program. Also modify the program to set icons for a component when it is selected, disabled, rollover or pressed.

14.6 RADIO BUTTONS

Radio Buttons are supported by the **JRadioButton** class, which is a concrete implementation of **AbstractButton**. Some of its constructors are :

JRadioButton(Icon <i>i</i>)	JRadioButton(Icon <i>i</i> , boolean <i>state</i>)
JRadioButton(String <i>s</i>)	JRadioButton(String <i>s</i> , boolean <i>state</i>)
JRadioButton(String <i>s</i> , Icon <i>i</i>)	JRadioButton(String <i>s</i> , Icon <i>i</i> , boolean <i>state</i>)

where *i* is the icon for the button, *s* specifies the text. If *state* is **true**, the button is initially selected, otherwise not.

Radio buttons should be configured in a group. As you know, only one button in that group can be selected at any time. If a user presses a radio button in a group, then any previously selected button gets deselected. In order to create a button group, we use the class **ButtonGroup**. Its default constructor is invoked to create the group and buttons are then added to the group with the method

```
void add(AbstractButton ab)
```

where *ab* is the reference to the button to be added to the group.

Radio button presses generate action events. In the following example, these events are handled by the **actionPerformed()** method. The **getActionCommand()** method gets the text associated with the button and uses it to set the text field.

```

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
/*
<applet code = "JRadio" width = 300 height = 50>
</applet>
*/

```

```

public class JRadio extends JApplet implements ActionListener {
    JTextField jtf;
    public void init() {
        Container cp = getContentPane();
        cp.setLayout(new FlowLayout());
        JRadioButton b1 = new JRadioButton("A");
        b1.addActionListener(this);
        cp.add(b1);
        JRadioButton b2 = new JRadioButton("B");
        b2.addActionListener(this);
        cp.add(b2);
        JRadioButton b3 = new JRadioButton("C");
        b3.addActionListener(this);
        cp.add(b3);
        ButtonGroup bg = new ButtonGroup();
        bg.add(b1);
        bg.add(b2);           // create a button group
        bg.add(b3);
        jtf = new JTextField(15);
        cp.add(jtf);
    }
    public void actionPerformed(ActionEvent ae) {
        jtf.setText(ae.getActionCommand());
    }
}

```

Study the program and its output.

14.7 COMBO BOXES

Swing provides a combination of a text field and a drop down list called a combo box through the **JComboBox** class, which extends **JComponent**. Normally a combo box displays one entry. But it can also display a drop down list which allows the user to select a different entry. Your own selection can also be typed in the text field.

Some constructors of **JComboBox()** are :

JComboBox() **JComboBox(Vector v)**

where *v* is a vector which initialises the combo box.

Items are added to the list using the **addItem()** method whose form is :

void addItem(Object obj)

where *obj* is the item to be added to the combo box.

Let us create a combo box and a label in the following example. The label displays an icon. The combo box contains entries from a menu list. Whenever an item is selected the label is updated to display the image of that item.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

/*
<applet code = "JCombo" width = 300 height = 50>
</applet>
*/

public class JCombo extends JApplet implements ItemListener {
    JLabel jl;
    ImageIcon home, travel, airport;

    public void init() {
        Container cp = getContentPane();
        cp.setLayout(new FlowLayout());
        JComboBox jc = new JComboBox();
        jc.addItem("Home");
        jc.addItem("Travel");
        jc.addItem("Airport");
        jc.addItemListener(this);
        cp.add(jc);
        jl = new JLabel(new ImageIcon("home.gif"));
        cp.add(jl);
    }

    public void itemStateChanged(ItemEvent ie) {
        String s = (String)ie.getItem();
        jl.setIcon(new ImageIcon(s + ".gif"));
    }
}
```

The output of the program is as shown below :



14.5 to 14.7 Check Your Progress.

1. Match the following.

Column A	Column B
a) Combo Boxes	i) a combination of a text field and a drop down list
b) Radio Buttons	ii) provided in the JCheckBox class
c) Check Boxes	iii) should be configured in a group.

14.8 TABBED PANES

A tabbed pane is a component which appears like a group of folders in a file cabinet. Each folder has a title. When a user selects a folder, its contents become visible. At any time, only one of the folders may be selected. Tabbed panes are mainly used to set configuration options. The **JTabbedPane** class encapsulates tabbed panes. It extends the **JComponent**.

```
void addTab(String str, Component comp)
```

is the method used to define tabs. Here *str* is the title for the tab and *comp* is the component that should be added to the tab. Typically a **JPanel** or its subclass is added.

Following is the procedure to use a tabbed pane in an applet :

- Create a **JTabbedPane** object
- Call **addTab()** to add a tab to the pane.
- Repeat the above step for each tab
- Add the tabbed pane to the content pane of the applet.

In the following example we shall create three tabs in a tabbed pane. The first tab is titled "Colors" and contains three check boxes. The second tab contains buttons to show names of flowers and the tab is titled "Flowers". The third tab titled "Sports" contains a combo box, which allows to select a sport from one of the three sports.

```
import javax.swing.*;

/*
<applet code = "JTabbed" width = 300 height = 50>
</applet>
*/

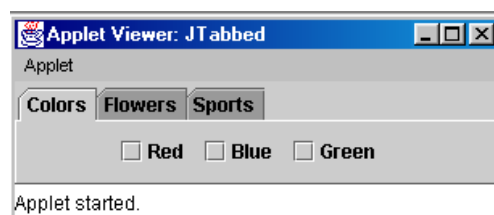
public class JTabbed extends JApplet {
    public void init() {
        JTabbedPane jp = new JTabbedPane();
        jp.addTab("Colors", new ColorPanel());
        jp.addTab("Flowers", new FlowersPanel());
        jp.addTab("Sports", new SportsPanel());
        getContentPane().add(jp);
    }
}
```

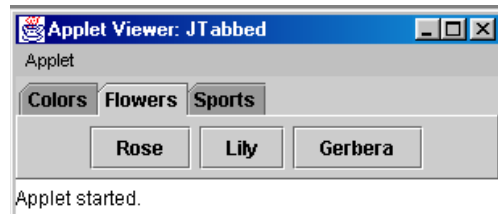
```

    }
}
class ColorPanel extends JPanel {
    public ColorPanel() {
        JCheckBox cb1 = new JCheckBox("Red");
        add(cb1);
        JCheckBox cb2 = new JCheckBox("Blue");
        add(cb2);
        JCheckBox cb3 = new JCheckBox("Green");
        add(cb3);
    }
}
class FlowersPanel extends JPanel {
    public FlowersPanel() {
        JButton b1 = new JButton("Rose");
        add(b1);
        JButton b2 = new JButton("Lily");
        add(b2);
        JButton b3 = new JButton("Gerbera");
        add b3;
    }
}
class SportsPanel extends JPanel {
    public SportsPanel() {
        JComboBox jc = new JComboBox();
        jc.addItem("Swimming");
        jc.addItem("Skating");
        jc.addItem("BasketBall");
        jc.addItem("Hockey");
    }
}
}

```

The output of the program is as follow :





14.9 SCROLL PANES

A scroll pane is a component that presents a rectangular area in which a component may be viewed. If necessary, horizontal and/or vertical scroll bars may be provided. Scroll panes are implemented using the **JScrollPane** class, which extends **JComponent**. Some of its constructors are :

`JScrollPane(Component comp)`

`JScrollPane(int vsb, int hsb)`

`JScrollPane(Component comp, int vsb, int hsb)`

where *comp* is the component to be added to the scroll pane, *vsb* and *hsb* are **int** constants that define when vertical and horizontal scroll bars for this scroll pane are shown. These constants are defined by the **ScrollPaneConstants** interface.

Some of these constants are :

Constant	Description
<code>HORIZONTAL_SCROLLBAR_ALWAYS</code>	always provide horizontal scroll bar
<code>VERTICAL_SCROLLBAR_ALWAYS</code>	always provide vertical scroll bar
<code>HORIZONTAL_SCROLLBAR_AS_NEEDED</code>	provide horizontal scroll bar if needed
<code>VERTICAL_SCROLLBAR_AS_NEEDED</code>	provide vertical scroll bar if needed

The steps to be followed to use a scroll pane in an applet :

- Create a **JComponent** object
- Create a **JScrollPane** object
- Add the scroll pane to the content pane of the applet.

The following example illustrates. We add a **JPanel** object to the scroll pane and the scroll pane is added to the content pane. We add buttons to the **JPanel** object. Vertical and horizontal scroll bars will appear if the buttons do not fit in the dimensions. These can be used to scroll the buttons into view.

```

import java.awt.*;
import javax.swing.*;
/*
<applet code = "JScrolls" width = 300 height = 50>
</applet>
*/
public class JScrolls extends JApplet {
    public void init() {
        Container cp = getContentPane();
        cp.setLayout(new BorderLayout());
        JPanel jp = new JPanel();
        jp.setLayout(new GridLayout(20,20));
        int b = 0;
        for(int i = 0; i < 5; i++) {
            for(int j = 0; j<5; j++) {
                jp.add(new JButton("Button " + b));
                b++;
            }
        }
        int v = ScrollPaneConstants.
VERTICAL_SCROLLBAR_AS_NEEDED;
        int h = ScrollPaneConstants.
HORIZONTAL_SCROLLBAR_AS_NEEDED;
        JScrollPane jsp = new JScrollPane(jp, v, h);
        cp.add(jsp, BorderLayout.CENTER);
    }
}

```

Run the program and study its output. Also try adding more buttons to see effect of the scrollbars.

14.10 TREES

A tree is a component which presents a hierarchical view of data. A user has the ability to expand or collapse individual subtrees in this display. Trees are implemented in **Swing** by the **JTree** class which extends **JComponent**. Some of its constructors are :

```
JTree(Hashtable ht)
```

This form creates a tree in which each element of the hashtable *ht* is a child node.

`JTree(Object obj[])`

In this form each element of the array *obj* is a child node.

`JTree(TreeNode tn)`

The tree node *tn* is the root of the tree in this form.

`JTree(Vector v)`

Elements of vector *v* are used as child nodes.

A **Jtree** object generates events whenever a node is expanded or collapsed. The **addTreeExpansionListener()** and **removeTreeExpansionListener()** methods allow listeners to register and unregister for these notifications.

The signatures of these methods are :

`void addTreeExpansionListener(TreeExpansionListener tel)`

`void removeTreeExpansionListener(TreeExpansionListener tel)`

where *tel* is the listener object.

The **getPathForLocation()** method is used to translate a mouse click on a specific point of the tree to a tree path. Its signature is :

`TreePath getPathForLocation(int x, int y)`

where *x* and *y* are the coordinates at which the mouse is clicked. The return value is a **TreePath** object that encapsulates information about the tree node that was selected by the user. This class provides several constructors and methods. We shall use only the **toString()** method which returns the string equivalent of the tree path.

The **TreeNode** interface declares methods that obtain information about a tree node eg. it is possible to obtain a reference to the parent node or the number of child nodes. The **MutableTreeNode** is an interface that extends **TreeNode**. It declares methods which can insert and remove child nodes or change the parent node. The **DefaultMutableTreeNode** class implements the **MutableTreeNode** interface. It represents a node in a tree.

One of its constructors is :

`DefaultMutableTreeNode(Object obj)`

where *obj* is the object enclosed in this tree node. The new tree node does not have a parent or children.

In order to create a hierarchy of tree nodes, there is a method **add()** in **DefaultMutableTreeNode**.

Its signature is :

`void add(MutableTreeNode child)`

child is a mutable tree node that is to be added as a child to the current node.

The **TreeExpansionEvent** class in the **javax.swing.event** package describes the tree expansion events. **getPath()** method of this class returns a

TreePath object which describes the path to the changed node.

The **TreeExpansionListener** interface provides the following methods :

`void treeCollapsed(TreeExpansionEvent tee)`

`void treeExpanded(TreeExpansionEvent tee)`

tee is the tree expansion event. The first method is called when a subtree is hidden and the second method is called when a subtree becomes visible.

The steps to use a tree in an applet are :

- Create a **JTree** object
- Create a **JScrollPane** object
- Add the tree to the scroll pane
- Add the scroll pane to the content pane of the applet.

The following example illustrates. In this example a **DefaultMutableTreeNode** labeled "TopMost" is created. This is the top node in the hierarchy. Additional tree nodes are created and connected to this node with the **add()** method. A reference to the top node is provided as an argument to the **JTree** constructor. The **JTree** is then provided as argument to **JScrollPane** constructor which is then added to the applet. A textfield is also created and added to the applet to present information about mouse click. For the purpose of receiving mouse events, the **addMouseListener()** method of the **JTree** object is called. The **doMouseClicked()** method processes mouse clicks. It calls the **getPathForLocation()** which translates the coordinates of the mouse click into a **TreePath** object.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import javax.swing.tree.*;
/*
<applet code = "JTrees" width = 400 height = 200>
</applet>
*/
public class JTrees extends JApplet {
    JTree tree;
    JTextField jtf;
    public void init() {
        Container cp = getContentPane();
        cp.setLayout(new BorderLayout());
//Create top node of tree
        DefaultMutableTreeNode top = new
```

```

DefaultMutableTreeNode("Topmost");
//Create subtree A
        DefaultMutableTreeNode a = new
        DefaultMutableTreeNode("A");
            top.add(a);
        DefaultMutableTreeNode a1 = new
        DefaultMutableTreeNode("A1");
        a.add(a1);
        DefaultMutableTreeNode a2 = new
        DefaultMutableTreeNode("A2");
        a.add(a2);
        DefaultMutableTreeNode a3 = new
DefaultMutableTreeNode("A3");
        a.add(a3);
//Create subtree B
        DefaultMutableTreeNode b = new
        DefaultMutableTreeNode("B");
        top.add(b);
        DefaultMutableTreeNode b1 = new
        DefaultMutableTreeNode("B1");
        b.add(a1);
        DefaultMutableTreeNode b2 = new
        DefaultMutableTreeNode("B2");
        b.add(a2);
        DefaultMutableTreeNode b3 = new
        DefaultMutableTreeNode("B3");
        b.add(a3);
        tree = new JTree(top);

// Create tree
        int v = ScrollPaneConstants.
VERTICAL_SCROLLBAR_AS_NEEDED;
        int h = ScrollPaneConstants.
HORIZONTAL_SCROLLBAR_AS_NEEDED;
        JScrollPane jsp = new JScrollPane(tree, v, h);
        cp.add(jsp, BorderLayout.CENTER);
        jtf = new JTextField(" ", 20);
        cp.add(jtf, BorderLayout.SOUTH);
        tree.addMouseListener(new

```

```

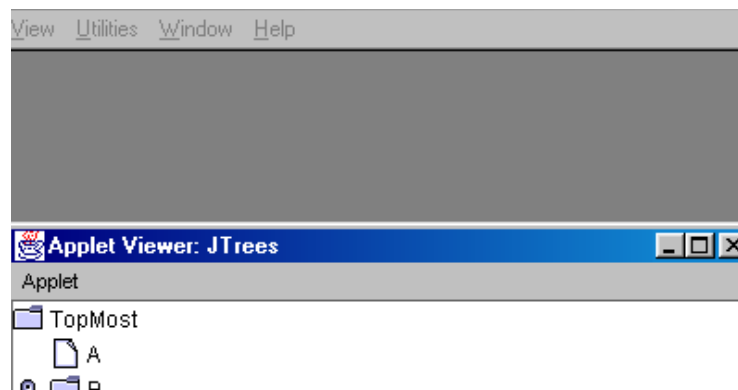
        MouseAdapter() {
        public void mouseClicked(MouseEvent me) {
                doMouseClicked(me) ;
        }
        });
    }

    void doMouseClicked(MouseEvent me) {
        TreePath tp =
        tree.getPathForLocation(me.getX(),
        me.getY());

        if(tp != null)
            jtf.setText(tp.toString());
        else
            jtf.setText(" ");
    }
}

```

output



14.11 TABLES

A table is a component which displays rows and columns of data. You can resize the columns by dragging the cursor on the column boundaries. You can also drag a column to a new position. Tables are implemented in the **JTable** class which extends **JComponent**. One of its constructors is :

```
JTable(Object data[], Object colHeads[])
```

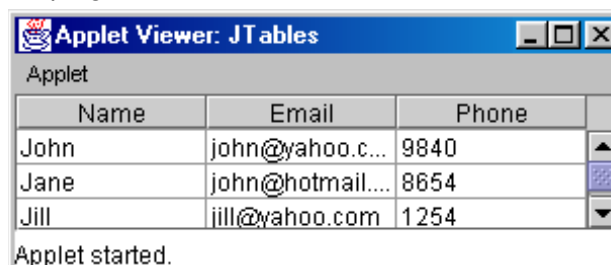
where *data* is a two dimensional array of the information to be presented and *colHeads* is a one dimensional array with the column headings. The steps for using a table in an applet are :

- Create a **JTable** object.
- Create a **JScrollPane** object.
- Add the table to the scroll pane.
- Add the scroll pane to the content pane of the applet.

Let us see how the following example implements a table in the applet. A one dimensional array of headings is created and a two dimensional array of data is created. The arrays are passed to the **JTable** constructor. The table is added to the scroll pane and the scroll pane is added to the content pane.

```
import java.awt.*;
import javax.swing.*;
/*
<applet code = "JTables" width = 300 height = 100>
</applet>
*/
public class JTables extends JApplet {
    public void init() {
        Container cp = getContentPane();
        cp.setLayout(new BorderLayout());
        final String[] colHeads = {"Name", "Email",
"Phone");
        final Object[][] data = {
            { "John", "john@yahoo.com", "9840"},
            { "Jane", "jane@hotmail.com", "8654"},
            { "Jill", "jill@yahoo.com", "1254"},
            { "Jack", "jack@yahoo.com", "6534"},
            { "Jerry", "jerry@hotmail.com", "7433"},
            { "Joseph", "joseph@hotmail.com", "4343"},
        };
        JTable table = new JTable(data, colHeads);
        // create table
        int v = ScrollPaneConstants.
VERTICAL_SCROLLBAR_AS_NEEDED;
        int h = ScrollPaneConstants.
HORIZONTAL_SCROLLBAR_AS_NEEDED;
        JScrollPane jsp = new JScrollPane(table, v, h);
        // add table to scroll pane
        cp.add(jsp, BorderLayout.CENTER);
        // add scroll pane to contentpane
    }
}
```

The output of the program is :



14.8 to 14.11 Check Your Progress.**1. Match the following.****Column A**

- a) Trees
- b) Tables
- c) Tabbed pane
- d) Scroll pane

Column B

- i) a component that presents a rectangular area
- ii) component which appears like group of folders
- iii) displays rows and columns of data
- iv) component which presents hierarchical view of data

14.12 SUMMARY

Swing is a set of classes which provides more flexible and powerful components than those that are possible in the AWT. Fundamental to Swing is the JApplet class which extends Applet.

A tabbed pane is a component which appears like a group of folders in a file cabinet. A tree is a component which presents a hierarchical view of data. A user has the ability to expand or collapse individual subtrees in this display. The TreeNode interface declares methods that obtain information about a tree node eg. it is possible to obtain a reference to the parent node or the number of child nodes.

Source : sasajava.blogspot.com (Link)

14.13 CHECK YOUR PROGRESS - ANSWERS**14.1 & 14.2**

- 1. a) True
- b) False
- c) False

14.3 & 14.4

- 1. a) JTextField
- b) AbstractButton
- c) rollover

14.5 to 14.7

- 1. a) - (i)
- b) - (iii)
- c) - (ii)

14.8 to 14.11

- 1. a) - (iv)

- b) - (iii)
- c) - (ii)
- d) - (i)

14.14 QUESTIONS FOR SELF - STUDY

1. Describe some methods of the **AbstractButton** class. Write a few constructors of the **JButton** class.
2. Describe the constructors of the **JCheckBox** class and the **JRadioButton** class. Which is the method used to add buttons to a group?
3. What is the meaning of a combo box? Write constructors of the combo box.
4. What is a tabbed pane? Write the procedure to add a tabbed pane to an applet.
5. What is a scroll pane? Describe the constructors of the **JScrollPane** class. Write down the steps to use **scroll** panes in an applet.
6. Write a detailed note on tree component of **Swing**.
7. What is the table component? Describe its constructors. Write the steps for using a table in an applet.
8. **Write short notes on :**
 - a) Icons in Swing
 - b) Labels in Swing
 - c) TextFields in Swing

14.15 SUGGESTED READINGS

1. www.java.com
2. www.freejavaguide.com
3. www.java-made-easy.com
4. www.tutorialspoint.com
5. www.roseindia.net



[illegible]